

Hybrid Memory Aware Cache Replacement Policy Design

Prateek, Roll no: 2101151

Advisor: Dr. Arijit Nath

Indian Institute of Information Technology, Guwahati

November 2024

Introduction

The rapid advancements in data-intensive applications have driven a demand for efficient and high-capacity memory systems. Hybrid memory systems integrate both DRAM and Non-Volatile memory.

- **DRAM** - fast access times energy efficient but limited capacity.
- **NVM** - larger storage capacity but slower write speeds and limited write endurance.

In hybrid memory systems, the cache replacement policy plays a crucial role in balancing these contrasting characteristics, aiming to maintain high performance, extend NVM longevity, and improve energy efficiency. Traditional replacement policies such as Least Recently Used (LRU) are effective for homogeneous memory systems but fall short in hybrid systems, where the difference between DRAM and NVM demands more nuanced management of cached data.

Literature Survey

- NVM-Based Architecture for Intermittent Computing by Y. Zhang et al. focuses on systems operating in environments with fluctuating power availability. It integrates NVM to retain the system's state during power loss, mitigating data loss by backing up data to non-volatile storage.[1]
- HAP: Hybrid-Memory-Aware Partition in Shared Last-Level Cache by Y. Zhang et al. addresses cache management in hybrid DRAM-NVM systems by dynamically partitioning cache space.[2]
- MALRU: The Memory Access Locality-aware Replacement Policy paper introduces a novel cache replacement strategy that enhances cache performance by prioritizing blocks with high memory access locality. By adapting replacement decisions to memory access patterns, MALRU reduces cache misses, improving efficiency in hybrid memory systems.[3]

Proposed Solution

- Separate replacement policies for DRAM and NVM.
- NVM to handle cold data, focusing on reducing writebacks (e.g., wear-leveling, write minimization strategies).
- DRAM to handle hot data focusing on improving hit rates(e.g. LRU)

We propose 2 cache replacement policies built over LRU - P1 and P2.

Use ChampSim to simulate real life conditions validating the performance of these policies.

Implementation

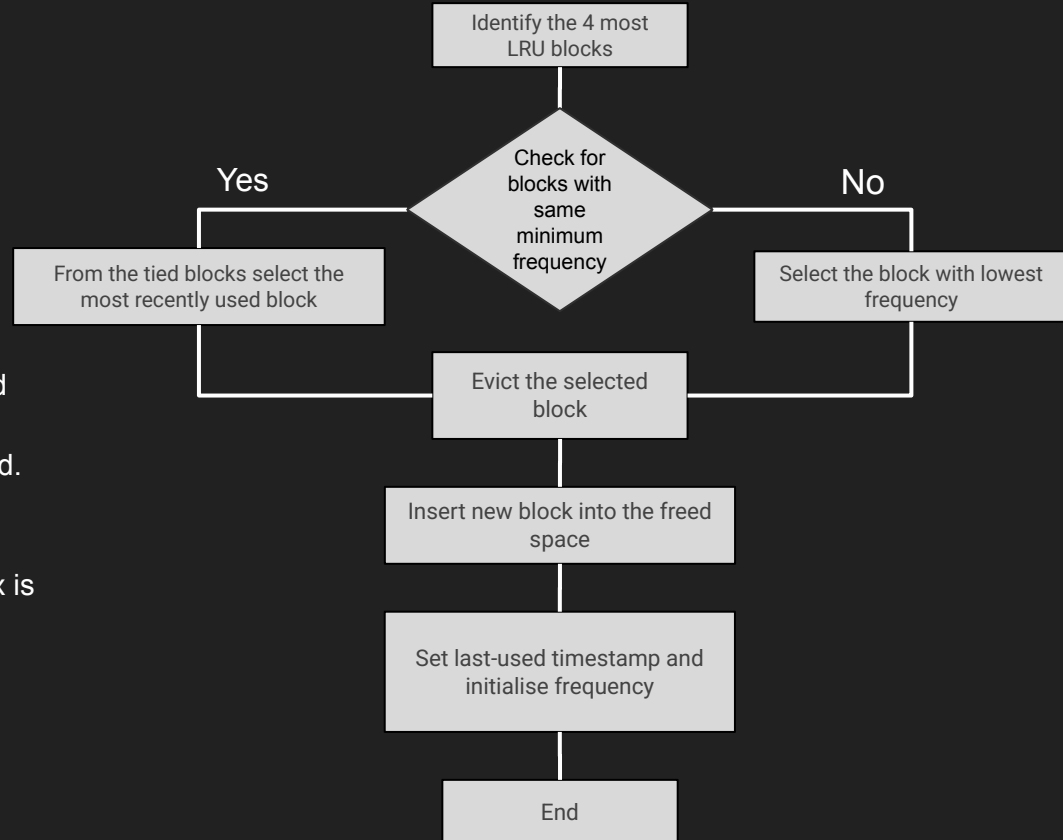
- **Baseline LRU** - Standard LRU
- **Policy P1**: Combines recency and frequency to prioritize frequently accessed blocks.
- **Policy P2**: Adds channel prioritization, evicting blocks from DRAM channels first, reducing wear on NVM.

Cache Block Identification and Mapping : Memory access generates an address, which maps to a specific cache set. The memory address is used to determine the channel and map the address to the cache, with blocks being tracked and managed by their memory channels.

Replacement policy flow

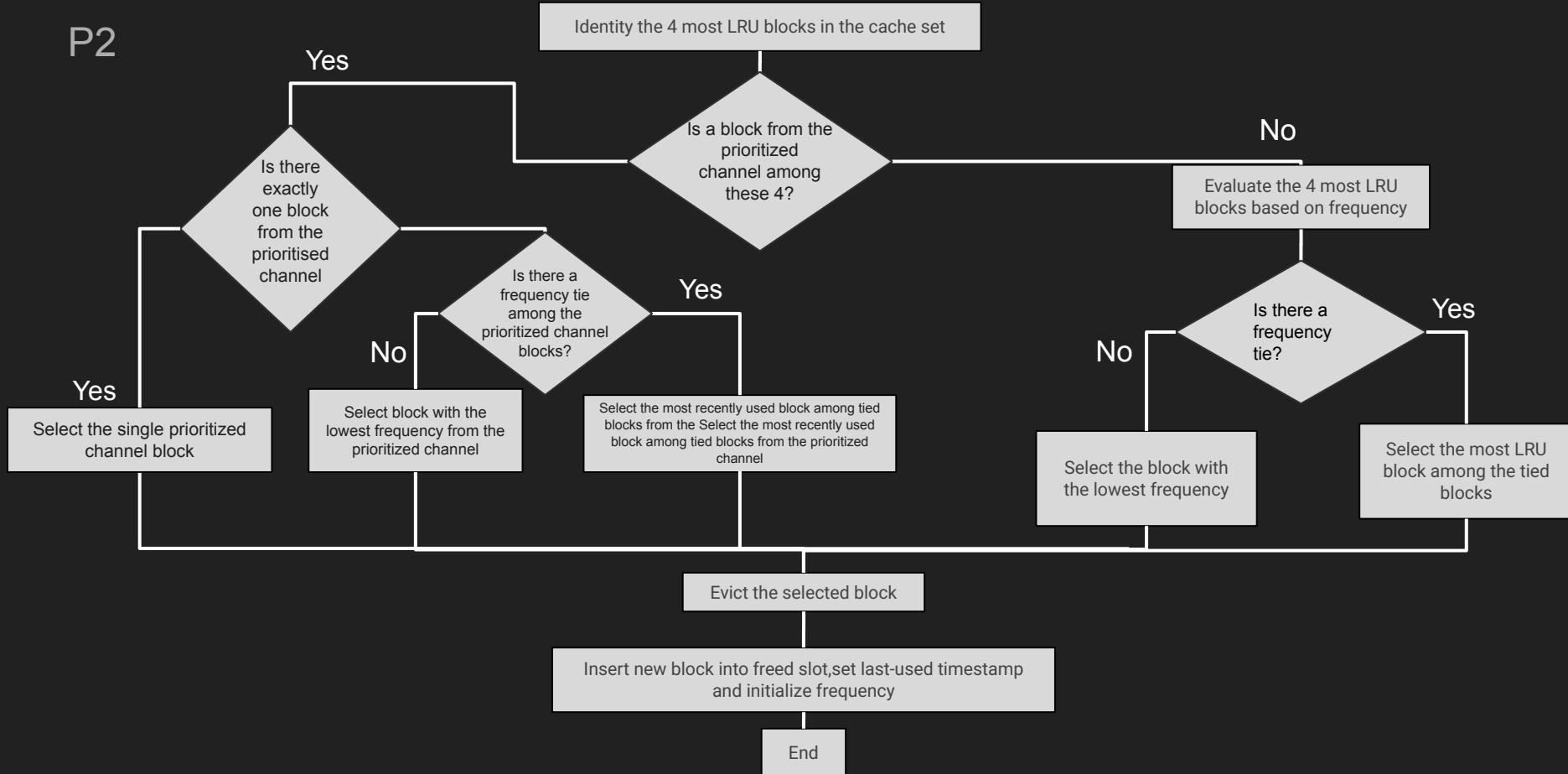
P1

- **Access Cache State:** Get the last access cycles (`last_used_cycles`) and access frequencies (`access_frequencies`) for the blocks in the set.
- **Track LRU:** Sort the first 4 blocks by their last access cycle to maintain the LRU order.
- **Determine Victim:** Compare the access frequency and cycle time for the top 4 LRU blocks:
 - The block with the lowest frequency is prioritized.
 - In case of ties, the block with the oldest access cycle is chosen.
- **Return Victim Index:** The selected victim block's index is returned for eviction.



Replacement policy flow

P2



P2 Implementation

1. **Access Cache State:** Get the last used cycles (`last_used_cycles`), access frequencies (`access_frequencies`), and channel IDs (`channel_ids`) for the blocks in the set.
2. **Track Victim Candidates:**
 - `victim_index`: Default index (not used in final selection).
 - `dram_index`: To store the index of the first DRAM block (Channel 0) found.
 - `lowest_freq_index` and `lowest_freq`: To track the block with the lowest access frequency.
3. **Initialize LRU Arrays:** Store the indices and access cycles of the cache blocks.
4. **Sort by LRU:** Sort the blocks based on their access cycles, from least recently used to most recently used.
5. **Loop Through LRU Blocks:**
 - For each block, check its channel (DRAM or NVM).
 - If it's a DRAM block (Channel 0), prioritize it for eviction.
 - Track the block with the lowest frequency, whether it's DRAM or not.
6. **Eviction Decision:**
 - If a DRAM block is found, evict it.
 - If no DRAM block is found, evict the block with the lowest frequency.

Prerequisites

ChampSim Simulator -

- Understand program flow and codebase of simulator.
- Understand channel and block routing for channel extraction
- Understanding default cache configuration.
- Understanding default cache replacement policies.

System Formulation

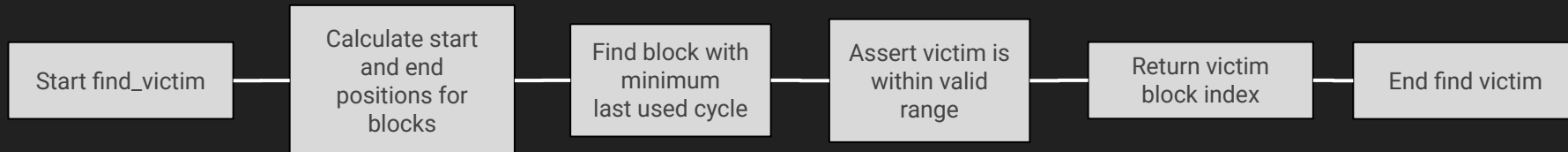
This study employs ChampSim, a trace-based simulator, to investigate cache performance improvements in a hybrid memory system. The system comprises 4 memory channels. The main goal is to tailor cache replacement policies based on the memory type of each accessed block to optimize cache performance.

- Executable Name: champsim
- Cache Block Size: 64 bytes
- Cache Hierarchy: LLC: 2048 sets, 16 ways, 20 cycles latency
- Memory Configuration: 4 Channels, 1 rank, 8 banks, 65536 rows, 128 columns
- Baseline replacement policy - LRU

Implementation

The baseline policy uses LRU. Two proposed policies (P1 and P2) were developed. P1 combines LRU and frequency considerations, while P2 introduces victim prioritization based on the channel a block belongs to.

Baseline Policy flow -



Methods used -

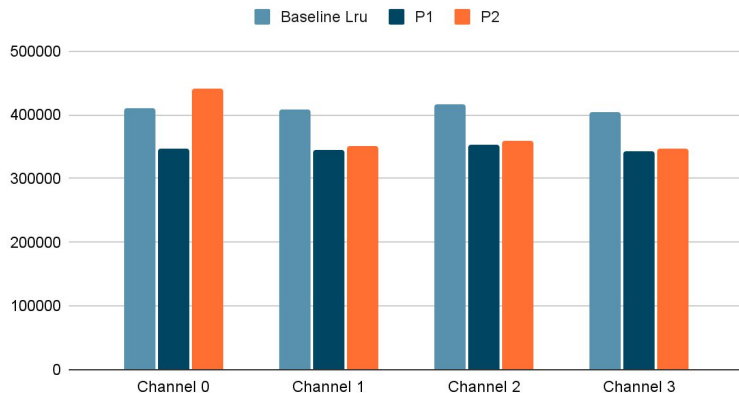
- find_victim
- update_replacement_state
- dram_get_channel

Experimental Results

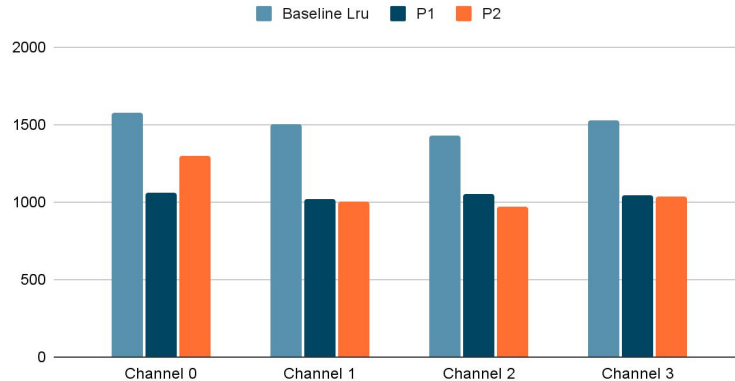
Writeback

Channel 0	Baseline Lru	P1	P2
Trace 1	1577	1062	1297
Trace 2	409808	347467	441733
Trace 3	24977	21695	30416

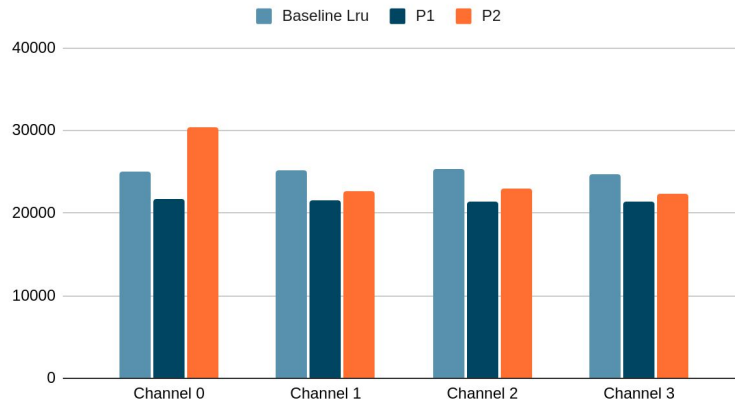
Trace 2



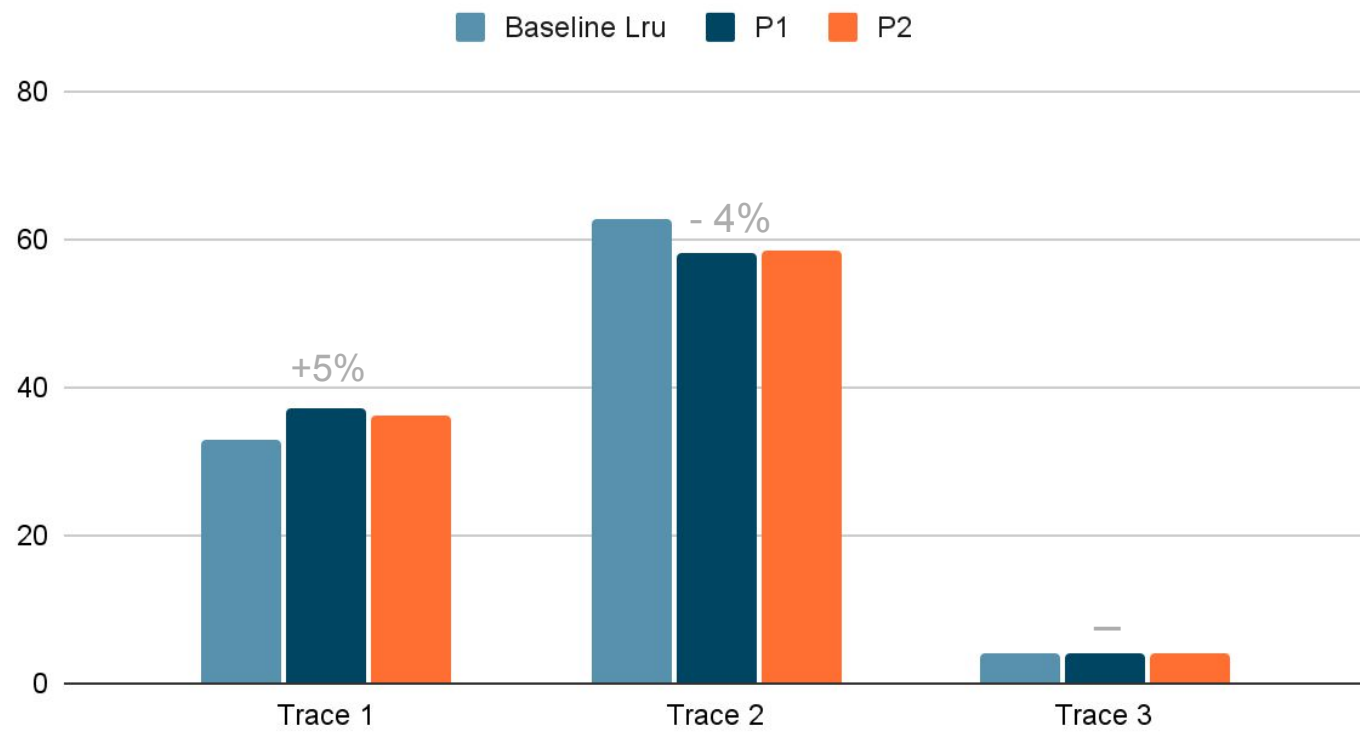
Trace 1



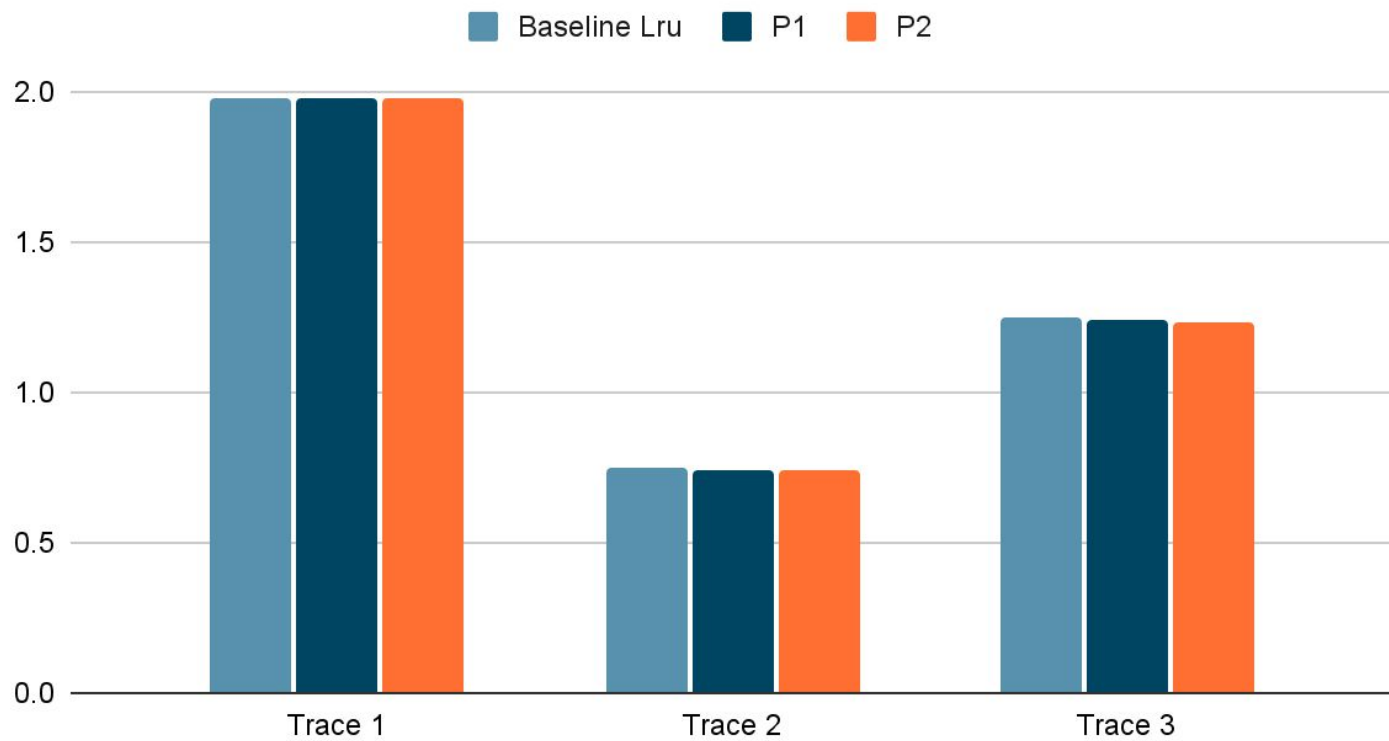
Trace 3



LLC hit rate



IPC



Result Compilation

- Writebacks - We can see that the writebacks reduce significantly in P1. When we apply P2 with channel prioritization we get to see increase in the writebacks for the channel prioritised.
- Hit rate - The LLC hit-rate doesn't experience significant changes, in some traces it increases and in some traces it decreases as it is sensitive to the characteristics of the trace and memory access patterns
- IPC - The ipc values remain more or less similar throughout the simulation.

Impact : Maintained cache performance, reduced writebacks to NVM and increased writebacks to DRAM.

Conclusion and future work

This project focused on enhancing the efficiency of cache management in hybrid memory systems that incorporate both DRAM and NVM. The tailored eviction policies led helped in decreasing memory congestion and optimizing bandwidth and utilization better memory management reducing unnecessary cache accesses and improving system efficiency.

In conclusion, the proposed hybrid eviction strategies for DRAM and NVM helped enhance cache performance, showcasing the potential of adaptive policies in reducing memory latency and optimizing memory system utilization.

The future of this project will focus on further optimization of policies and a deeper study of the ChampSim simulator. We will look at more components that would help us get better results and optimize our approach better.

References

1. Y. Zhang, et al., "NVM-Based Architecture for Intermittent Computing," *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019, pp. 1-12.
2. Y. Zhang, et al., "HAP: Hybrid-Memory-Aware Partition in Shared Last-Level Cache," *Proceedings of the 26th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2020, pp. 213-225.
3. Y. Zhang, et al., "MALRU: The Memory Access Locality-aware Replacement Policy," *Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2021, pp. 345-356.